第 11 章 VibeCoding 实战：能做什么，以及人该站在哪里

> 本章目标：把"AI 能帮我写代码"这句口号，拆成一张可对照的能力地图；认清 VibeCoding 真正的产出不是代码而是**可验证的系统**；并在课程结尾讨论一个人与 AI 分工的稳定位置。

前几章讲的是零件：模型、token、Agent、上下文、验证。这一章把它们装成一台机器，然后回答两个问题：VibeCoding 到底能做什么？人在这个过程中还剩什么？

11.1 一张能力地图

先给结论：VibeCoding 已经能覆盖"中等规模、单人或小组、有明确验收标准"的软件任务。下面这张表来自真实做过的项目，按"难度来源"而不是按"行业"分类，因为决定难度的从来不是行业。

类型	例子	难度真正来自哪里	典型证据
纯前端页面	个人主页、作品集、活动页	审美与信息层次，不是代码	打开就能看、换设备不塌
带状态的应用	笔记软件、知识库、聊天应用	数据模型与边界情况	刷新后数据还在、越权访问被拒
网页 + 硬件	机械臂工作台、随动云台、无人机地面站	实时性、断连、安全限位	断网会停、越界会被拦
游戏与模组	沙盒 demo、FPS 靶场、游戏 Mod	手感与规则一致性	玩十分钟不出错、存档能恢复
脚本与自动化	批量改文件、数据清洗、爬取整理	异常输入	坏数据不会让整批任务崩掉
仿真与计算	物理仿真、公式推导、数据分析	模型是否正确，不是代码是否跑通	与已知结果对得上
文本生产	论文草稿、小说、文案、角色扮演	事实与风格，不是字数	引用真实存在、风格前后一致
工具型机器人	QQ 群机器人、告警、定时任务	权限与滥用防护	普通用户不能触发管理命令

这张表最重要的信息在第三列：**几乎所有项目的真正难点都不在"写代码"**。代码是 Agent 最擅长的部分，而验收标准、边界情况、安全约束、审美判断，才是决定项目能不能用的部分。

反过来，也有明确做不好的：需要长期运行的稳定性保障、需要专门硬件认证的场合、需要承担法律责任的判断、需要跟真实对手持续对抗的系统。这些不是"模型不够强"，而是**错误的代价高到必须由流程和组织来吸收**。

11.2 三种用法，产出完全不同

同一个 Agent，用法不同，结果差一个数量级。实践中可以分成三种：

第一种：让它写完。 你给需求，它给成品。适合一次性、低风险、验收标准明确的东西——一个静态页面、一个数据清洗脚本、一个 demo。这种用法效率最高，也最容易高估自己：你拿到的是"能跑"，不是"能维护"。

第二种：让它帮你理解。 你读不懂一段代码、一个报错、一篇论文，让它逐段解释、举反例、画流程。这种用法几乎无风险，收益却常被低估。**读懂别人的系统，是改对它前提。**

第三种：让它帮你检查。 你写完，让它找问题：边界条件、并发、权限、遗漏的分支。这类用法要求你自己先有判断力——它给出的十条意见里可能三条是错的，你得能挑出来。

> 要点：新手最容易只用第一种；而实际项目里，第二、第三种用法的总时间往往超过第一种。

一个具体的对照：机械臂项目里"把跟随颜色从红改成绿"，属于第一种，改一个参数，几分钟。而"为什么拖动滑条结束后机械臂还在动"属于第二种加第三种——得先读懂请求合并的逻辑，再检查是不是合并窗口设错了。前者 Agent 一次就对，后者要人来判断"哪里算结束"。

11.3 从想法到交付：一条能复用的路径

不管做什么项目，下面这条路径都成立。它可以当作整门课的浓缩版。

1. **写清用户与场景。** 谁在什么情况下用它？不用写"面向所有人"。
2. **列出三个核心功能与验收条件。** 注意是三个，不是十个。每个功能配一句"怎样算做到了"。
3. **先做一条最窄的完整链路。** 从输入到输出全程打通，哪怕很丑。只有界面没有数据的项目，永远做不完。
4. **跑起来，找真人试用。** 自己用和给别人用是两件事，卡住的地方往往是你没想到的。

5. **修真实问题，不加新功能。** 这一周只修，不扩。绝大多数项目死在这一步的反面。
6. **补边界：空输入、坏数据、断网、越权、重复点击。**
7. **写验收证据。** 截图、日志、测试输出、真人试用记录。
8. **交付：源码、说明、已知问题。** 已知问题不是减分项，隐瞒才是。

第 3 步和第 5 步是分水岭。做不出第 3 步的人，通常一直在做界面；做不到第 5 步的人，功能越加越多，没有一个是稳的。

11.4 三个真实片段

片段一：无人机。 一开始的目标是"用 AI 写一个能飞的程序"。真实推进时发现，代码只占很小一部分，大头是：失控保护、低电量返航、地理围栏、地面站与飞控的协议对接、每次改动后的地面测试。Agent 能写出飞控通信代码，但**"什么情况下必须自动降落"这条规则必须由人定**，因为它是价值判断，不是技术选择。

片段二：游戏模组。 改一个老游戏的 Mod，看起来是纯代码任务。实际最花时间的是反查游戏的数据结构与资源格式，以及"改完之后存档还能不能读"。Agent 很擅长在你给出结构之后批量生成内容，但它不知道你的存档里有什么。**上下文不在代码里，在你的机器上。**

片段三：长篇文本。 用 AI 辅助写小说或论文，第一万字很惊艳，第三万字开始人物性格漂移、引用开始编造。这不是模型退化，而是**长程一致性需要一个外部的状态载体**——人物表、时间线、参考文献清单。这和第 9 章讲的 MEMORY 是同一件事：把该记住的东西写下来，而不是指望它记得。

三个片段的共同点：**Agent 放大你的工程能力，也放大你工程习惯的缺失。** 你有验收标准，它就快；你没有，它就飞快地做出一个错误的东西。

11.5 人是总的路由器

课程开头把 AI 拆成八级台阶，结尾要回答一个更根本的问题：人在这套系统里做什么？

一个我认为站得住的回答是：**人是路由器。** 路由器的职责不是计算，而是决定"这个请求该去哪"。

具体到日常，这个角色包含四件事：

- **决定谁来做。** 这件事该用规则写死、该查文档、该问人、还是该交给模型？把"写个正则"派给大模型，和把"判断这段合同有没有风险"派给正则，是同一种错误。
- **决定给多少上下文。** 给多了污染，给少了瞎猜。第 9 章整章都在讲这件事，而最终判断仍由人做。
- **决定信不信。** 模型的输出是一条线索，不是结论。信不信取决于证据等级，不取决于语气。

- **决定停不停。** 什么时候继续迭代，什么时候承认这条路走不通、换方案。这是 Agent 最不擅长的判断，因为它没有"够了"的感觉。

> 要点：AI 越强，"选对问题"和"判断做完了没有"这两件事越是人的专属。它们无法被外包，因为外包的前提是你能验收。

11.6 被 AI 忽悠，还是让 AI 听话

这两种体验的差别，技术上是同一个东西：**你有没有建立反馈回路**。

被忽悠的典型场景：你问一个事实，它答得流畅，你没查，就信了；你让它改代码，它说改好了，你没跑，就交了。让 AI 听话的典型场景：你把验收条件写清楚，它交付，你跑测试，不通过就把失败输出贴回去，直到通过。

所以"让 AI 听话"不是靠更强的提示词，而是靠**可检验的交付物**。这也是为什么整门课反复强调"跑得通 / 测得过 / 现场看得见"——它不是工程教条，它是你对抗忽悠的唯一工具。

一个实用技巧，常被叫做 **grilling**（拷问）：让 AI 先反过来问你。比如"我要做一个家庭聊天应用，你先问我十个问题，再给方案"。它问出来的问题，往往正是你没想清楚的地方。但注意它的局限：它只会问它"觉得重要"的问题，而你真正的约束（预算、家里老人的手机型号、你只有周末有时间）它猜不到。**拷问能补漏，不能替代你自己想。**

11.7 两个绕不开的问题

课程最后，有两个问题值得认真讨论，而不是用一句"AI 只是工具"打发掉。

第一个：人类知识是不是一个凸包？ 如果模型是在人类已有文本上训练的，它擅长的是已有知识之间的插值与组合，而不擅长跳到分布之外。这解释了两件事：为什么它写常规代码又快又好，为什么它在真正新颖的问题上会一本正经地胡说。对学生的实际含义是：**把 AI 用在"已有大量先例"的任务上，收益最大；用在"没有人做过"的地方，必须自己承担验证。**

第二个：机器人三定律成立吗？ 阿西莫夫的三定律是科幻设定，不是工程规范。真实系统里，安全靠的是限位、急停、权限、人工确认、物理隔离这些枯燥的东西，而不是写在提示词里的一句"不要伤害人类"。机械臂项目就是这个道理的具体版：让语言模型输出"向左转十度"，靠的不是模型懂事，而是角度上限、方向绑定表和执行前的预览确认。

> 要点：与其问"AI 会不会失控"，不如问"这个系统的失败模式是什么、谁来兜底"。前者是哲学，后者是工程，而后者能救命。

11.8 课程的终点

这门课不指望把你训练成 AI 专家。它希望你在结课时能做到六件事（第 13 章会给出可观察的验收方式）：

1. 区分模型、聊天产品、API、Agent 和普通程序，说清哪些在本地、哪些在云端；
2. 用至少两种模型服务完成同一件事，并说出差别；
3. 向 Agent 提交明确任务，看懂修改、跑通测试、保留版本、失败能回滚；
4. 亲手运行至少一种非 LLM 模型，说清输入、输出、适用范围与失败情况；
5. 发现一次不可靠输出，并用资料或实测核验；
6. 交付一个能运行的项目，完成一次临时变化任务，说清已知限制。

这六条没有一条要求你会训练模型。它们要求的都是同一件事：**在一个人机混合的系统里，做一个能负责的人。**

课后题

1. 从 11.1 的表里选一个类型，写出你自己的项目在这个类型下的"难度真正来自哪里"，以及对应的验收证据。
2. 用"三种用法"分析你最近一次使用 AI 的经历：当时用的是哪一种？如果换成另外两种，结果可能有什么不同？
3. 拿一个你想做的项目，按 11.3 的八步写出前四步，每步不超过三行。特别写下第 2 步的三个验收条件。
4. 找一个 AI 给过你、你当时信了、后来发现不对的答案。复盘：你当时缺的是哪一环——资料、测试、还是"停下来想一想"？
5. 思考题：如果 AI 只能做"人类已有知识的插值"，那么它在哪些任务上会长期强于人类？在哪些任务上可能永远不如一个认真的新手？举出各两个例子。
6. 思考题：有人说"AI 让每个人都成了项目经理"。你认为这个说法在什么条件下成立、在什么条件下不成立？结合你身边的例子说明。